examples_readme

SV-only starter scaffold

This is a **minimal plain-SystemVerilog** starting point that already satisfies the grading contract in `harness/grading_interface.md`. Copy the `sv_only/` tree into your submission directory, rename the tests, add coverage and assertions, and flesh out the reference model.

The scaffold deliberately does NOT use UVM. It is built around plain `module` / `program` constructs and a simple task-based reference model.

What is inside

```
sv_only/
  tb/
    tb_top.sv              # module top; instantiates dut_wrapper + BFMs
    apb_master_bfm.sv      # APB master BFM (write/read tasks)
    spi_slave_bfm.sv       # Simple SPI slave responder (MISO driver)
  env/
    ref_model.sv           # Predictor + scoreboard (SV classes, no UVM)
    coverage.sv            # Functional covergroups
  tests/
    sanity_test.sv         # Example directed test (mode 0, 1 byte)
    ral_hw_reset_test.sv   # Stub bonus test that prints TEST_SKIPPED
  sequences/
    stim_lib.sv            # Reusable randomisable transaction classes
  assertions/
    spi_sva.sv             # SVA module bound to u_dut.u_regfile / u_core
  Makefile                 # Concrete Makefile matching the template
```

How the test dispatcher works

`tb/tb_top.sv` reads `+TESTNAME=<name>` (or `+UVM_TESTNAME=<name>` as a fallback - the grader always passes both). The `case` inside `tb_top` forwards to the matching test program. Add one `case` arm for every new test you write; keep the exact name list in `Makefile:REGRESSION_TESTS` and in Section 3 of the grading contract.

Backdoor register access (SV-only version)

The DUT is split into `u_dut.u_regfile` and `u_dut.u_core`. Internal signals are directly hierarchically accessible from SV-only testbenches, which is the SV-only equivalent of UVM RAL backdoor access:

```
// Read CTRL.EN purely via backdoor
logic en_bd = dut_wrapper_inst.u_dut.u_regfile.ctrl_en;
// Peek the RX FIFO depth
int rx_depth = dut_wrapper_inst.u_dut.u_regfile.rx_count;
```

You do NOT qualify for the UVM RAL bonus by doing this - that bonus requires an actual `uvm_reg_block`. But backdoor checks in SV-only are still useful for catching sticky-interrupt bugs and FIFO bookkeeping bugs that the public APB interface cannot observe directly.

Bonus test stub

`tests/ral_hw_reset_test.sv` is a deliberate no-op that prints `[TEST_SKIPPED] ral_hw_reset_test`. The grader treats this as a zero score on the RAL bonus and does not penalise the base rubric. Delete the stub and replace it with a real UVM RAL test if you want the +5%.

Typical workflow

```
# Compile + run a single test against golden RTL
make -f Makefile run TEST=sanity_test SEED=1

# Full regression (200 runs by default = 10 tests * 20 seeds)
make -f Makefile regress

# Coverage report
make -f Makefile cov
```

Replace `make` with `gmake` on Windows if your `make` is Microsoft's nmake, or run from a MSYS2 / WSL shell.